

MetaGPT业务场景介绍

Atoms是什么？

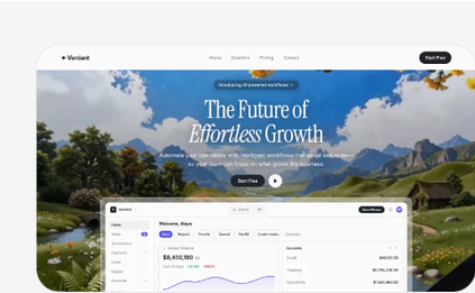
Atoms 是一个原生 AI 平台，可将创意转化为软件、内容和数字体验。智能体协同完成规划、设计、代码能力和执行，让团队在一个连续的工作流中从想法直达结果。



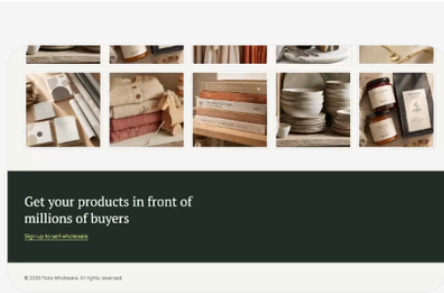
Atoms 将创意变成产品

从一个想法出发，完成调研、构建、上线并推动真实业务增长。

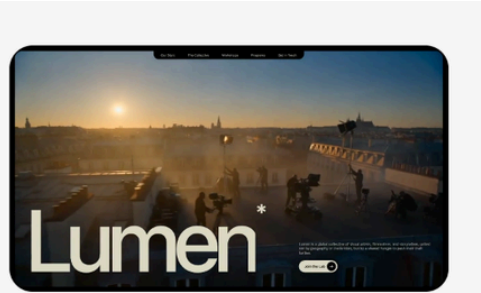
[了解 Atoms 如何工作](#)[开始使用](#)



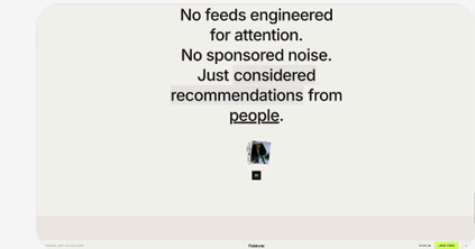
SaaS 落地页



独立品牌与批发



电影工作室网站



热门推荐平台



创意视频



射击游戏

展示一个流程



与CC和Codex的区别

Codex 和 Claude Code 本质上更偏 AI software engineer。它们非常适合已经有代码仓库、有明确 engineering task 的开发者，比如修 bug、做 refactor、实现 feature、跑测试、提 PR。它们优化的是 software engineering productivity。

Atoms 往上抽象了一层，想解决的不是“帮用户把代码写出来”，而是“帮用户把一个 idea 变成一个真正可以运行甚至商业化的 product”。

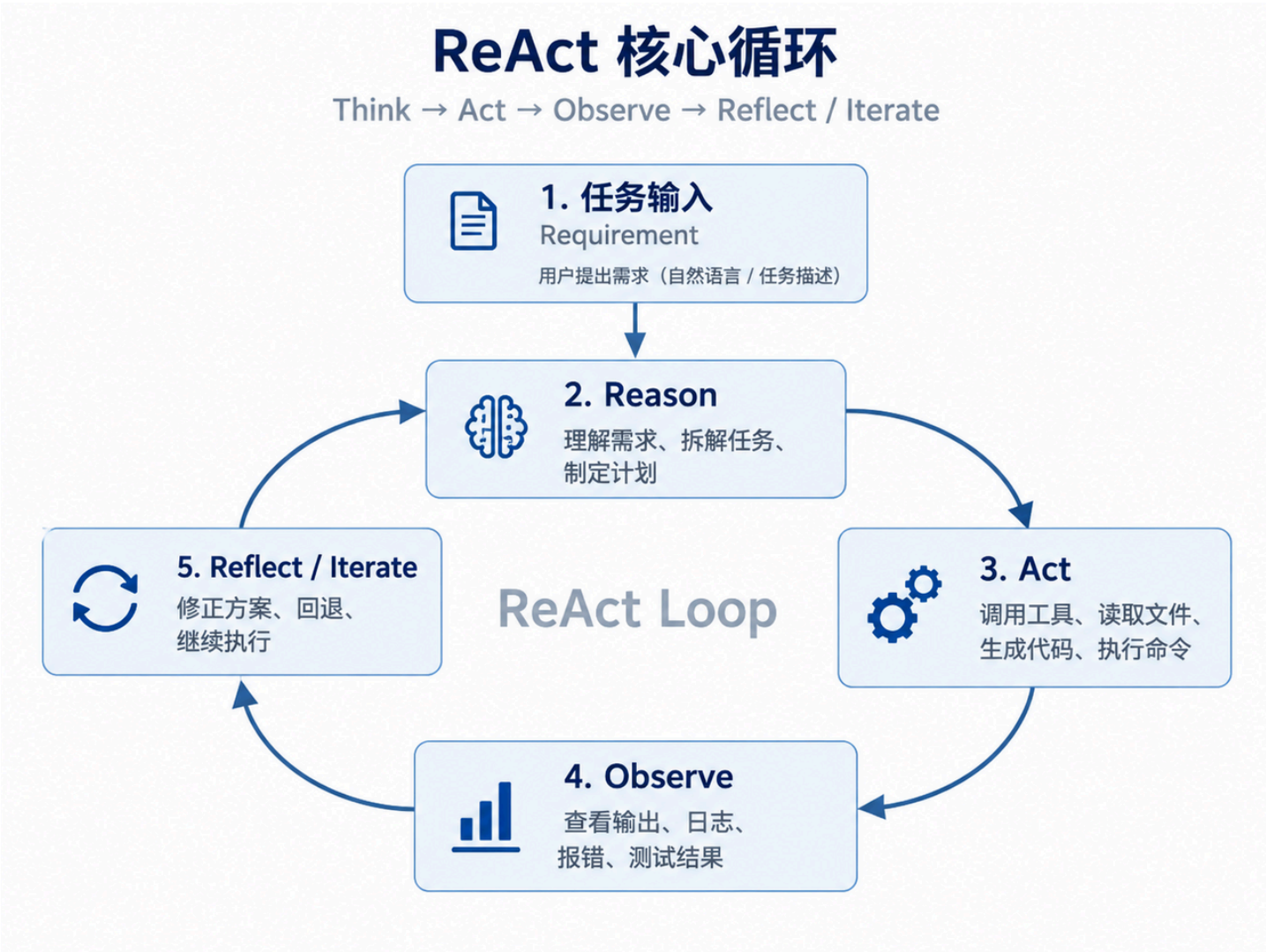
用户可以从一个很模糊的 business idea 开始，Atoms 里面不同角色的 agent 会去做 market research、竞品分析、产品定义、architecture、UI、前后端开发，再到 database、authentication、payment、deployment，甚至后面的 SEO 和增长。对用户来说，他管理的不是一个 coding agent，而更像是在管理一个 virtual startup team。

当然，最重要的区别在production infrastructure上。

对于一个 founder 来说，真正麻烦的往往不是生成一个 React 页面，而是 authentication、database、storage、payment、hosting 这些东西怎么连起来。Atoms Cloud 把这些能力做成了 agent 可以直接操作的 infrastructure，所以从 prototype 到一个真正能让用户注册、付费的产品，中间的 gap 会小很多。

Atoms的本质

Atoms 的本质可以理解为一个 **Coding Agent**：用户给出一个相对高层的产品或开发需求后，Agent 会自主完成需求拆解、开发规划、代码编写、工具调用、运行测试以及结果验证，并根据执行过程中遇到的问题不断调整方案，最终交付一个可以运行的软件产品。相比传统的代码补全工具，Atoms 更关注的是让 Agent 完成一整条软件开发任务，而不只是生成某一段代码。



一个项目的目录结构

就像一个github的repo一样，本质就是一个项目文件夹。

文件名	大小
 assets  	-
 app  	-
 docs  	-
 .atoms  	-
 .wiki.md 	0 KB

日志展示

MetaGPT 的 rollout.log 大概长什么样？

一个真实运行日志的简化展示：多 Agent 思考、工具调用、模板初始化、警告与异常回退

rollout.log

```
2026-06-02 17:01:25.450 | INFO | metagpt.const:get_metagpt_package_root:23 -  
Package root set to /Users/.../MetaGPT  
2026-06-02 17:01:28.994 | WARNING | metagpt.const:set_default_workspace_root:199 -  
update DEFAULT_WORKSPACE_ROOT from /Users/.../workspace to /Users/.../cases/digital-art-gallery  
2026-06-02 17:01:29.067 | INFO | metagpt.roles.di.role_zero:set_longterm_memory:533 -  
Long-term memory set for role 'Alex'  
2026-06-02 17:01:29.638 | INFO | metagpt.environment.mgx.mgx_env:publish_message:38 -  
User Requirement: create a digital art gallery website...  
2026-06-02 17:01:29.716 | INFO | metagpt.roles.di.role_zero:_think:875 - Alex, ready to think  
2026-06-02 17:01:38.889 | WARNING | metagpt.utils.cost_manager:update_cost:49 -  
Model claude-sonnet-4-6 not found in TOKEN_COSTS.  
2026-06-02 17:01:38.889 | INFO | metagpt.roles.di.role_zero:_think_background:892 -  
Command response:  
<FrontendEngine.draft_plan>  
1. Homepage with grid layout...  
2. Artwork detail page...  
...  
2026-06-02 17:01:42.046 | INFO | metagpt.tools.libs.be.backend_manager:enable_backend:182 -  
Backend funcsea enabled successfully  
HTTP error 401 for POST https://metagptx.dev.metadl.com/api/v1/oidc/internal/clients  
Command FrontendEngine.use_template executed: Traceback (most recent call last):  
  File "/Users/.../metagpt/tools/template.py", line 210, in use_template  
    client = OIDCClient(config)  
  File "/Users/.../metagpt/utlils/oidc.py", line 86, in __init__  
    raise HTTPError(f"401: {response.text}")  
HTTPError: 401: Unauthorized  
2026-06-02 17:01:47.818 | INFO | metagpt.roles.di.frontend_engine:use_template:531 -  
Template 'shadcn/ui' initialized  
2026-06-02 17:02:56.167 | INFO | metagpt.roles.di.role_zero:_act:925 -  
Command ImageCreator.generate_images executed: {'status': 'success', ...}
```

1 时间戳：记录每一步发生的时间

形如 2026-06-02 17:01:25.450，
用于定位和排查问题。

2 日志级别：常见有 INFO / WARNING

用不同的颜色区分日志级别，
方便快速识别重要信息。

3 模块路径：能看到具体组件、函数和行号

例如 metagpt.roles.di.role_zero:_think:875，
可以快速定位到相关代码。

4 Agent 行为：会出现 ready to think、
Command response、Commands outputs 等

能看到各个 Agent 的思考过程、
工具调用以及输出结果。

5 异常也会保留：例如 HTTP error、
Traceback、回退到其他方案

完整记录异常信息，便于问题排查，
也能看到后续的回退和继续执行。



典型特点：长路径、多模块名、Agent 思考过程、工具调用结果、以及警告/异常信息。